TDDはじめて物語

2016/2/27 TDDBC in Tokyo 2016-02

大中浩行

Copyright 2016 Hiroyuki Onaka

Generated by 社畜ちゃん台詞メーカー
http://blog.oukasoft.com/OS/

テスト駆動開発(Test Driven Development)

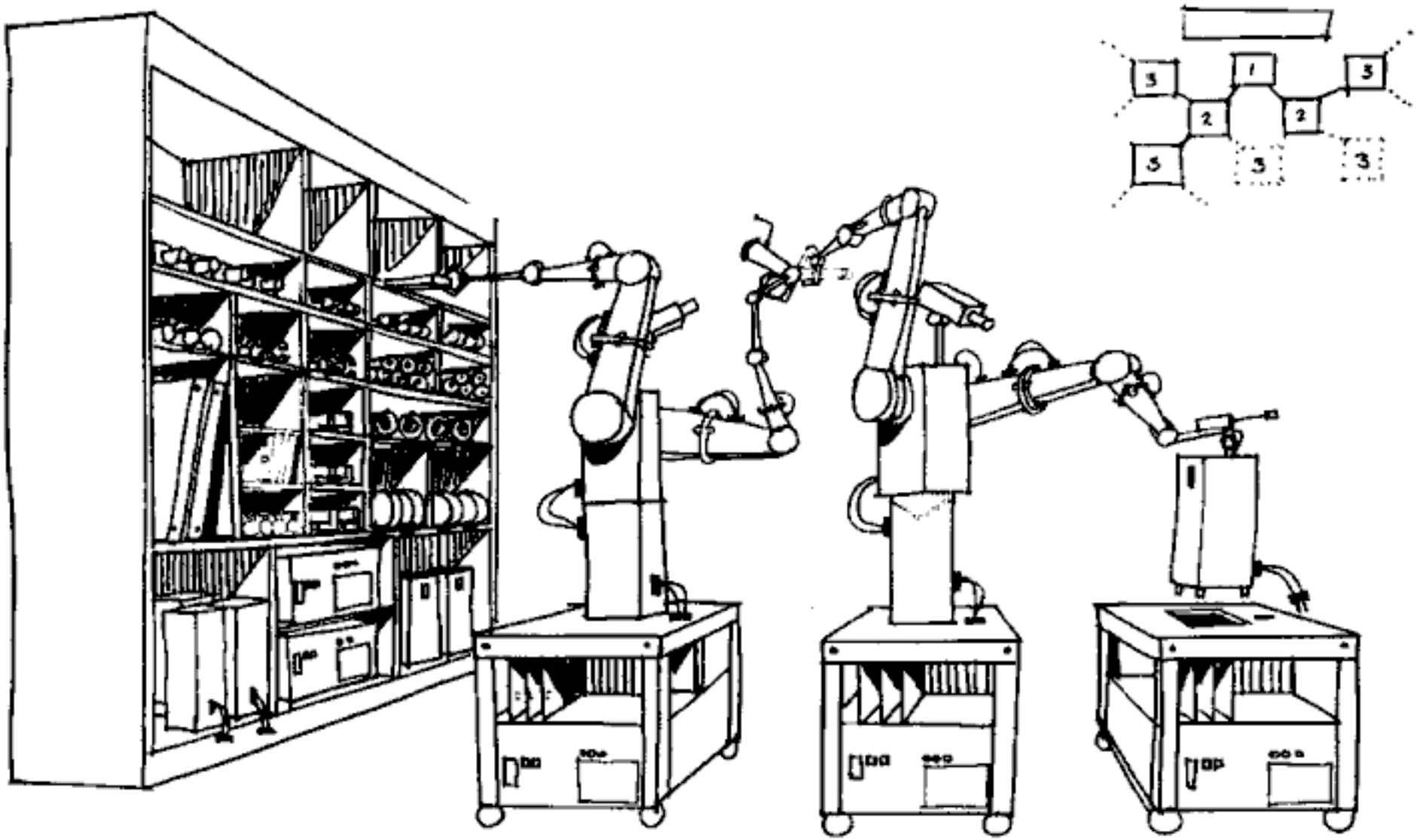
Copyright 2016 Hiroyuki Onaka

By National Photo Company [Public domain], via Wikimedia Commons
https://en.wikipedia.org/wiki/Bulletproof_vest

テストファーストしたら？

By NASA [Public domain], via Wikimedia Commons
https://en.wikipedia.org/wiki/Self-replicating_machine

テストが自動化されたら？



TDDとは

「TDDとは、プログラミングとテストすることを工程として統合した開発スタイルです。」

- @setoazusa

Copyright 2016 Hiroyuki Onaka

アジェンダ

- TDDとは
- テストファーストとは
- なぜTDDするのか
- TDDには何が必要か
- なぜTDDするのか(again)

Copyright 2016 Hiroyuki Onaka

TDDとは

Copyright 2016 Hiroyuki Onaka

TDDとは

テスト駆動開発入門

TDD: TEST-DRIVEN DEVELOPMENT BY EXAMPLE

KENT BECK

※絶版

By Improve It (Flickr: Kent Beck no Workshop Mapping XP.) [CC BY-SA 2.0 (http://creativecommons.org/licenses/by-sa/2.0)], via Wikimedia Commons
https://en.wikipedia.org/wiki/Kent_Beck

Kent Beck

TDDとは

1．素早くテストを追加する。

2．すべてのテストを実行し、新しいテストの失敗を確認する。

3．小さな修正を行う。

4．すべてのテストを実行し、すべての成功を確認する。

5．重複を取り除くためにリファクタリングを行う。

Copyright 2016 Hiroyuki Onaka

座標平面上にある点で、**x座標** (x-coordinate) 、**y座標** (y-coordinate) がともに整数である点を**格子点**と呼びます。

- **x座標**と**y座標**を与えて**格子点**を生成してください
- 生成した**格子点**から**x座標**と**y座標**を取得してください
- 生成した**格子点**から**文字列表記** (notation) を取得してください

Copyright 2016 Hiroyuki Onaka

テストを追加する。

```
@RunWith(JUnit5.class)
public class GridPoinTest {

    @Test
    public void x座標とy座標を与えて格子点を生成できること(){
}

    @Test
    public void 生成した格子点からx座標とy座標を取得できること(){
}
    @Test@Disabled
    public void 生成した格子点から文字列表記を取得できること() {}
}
```

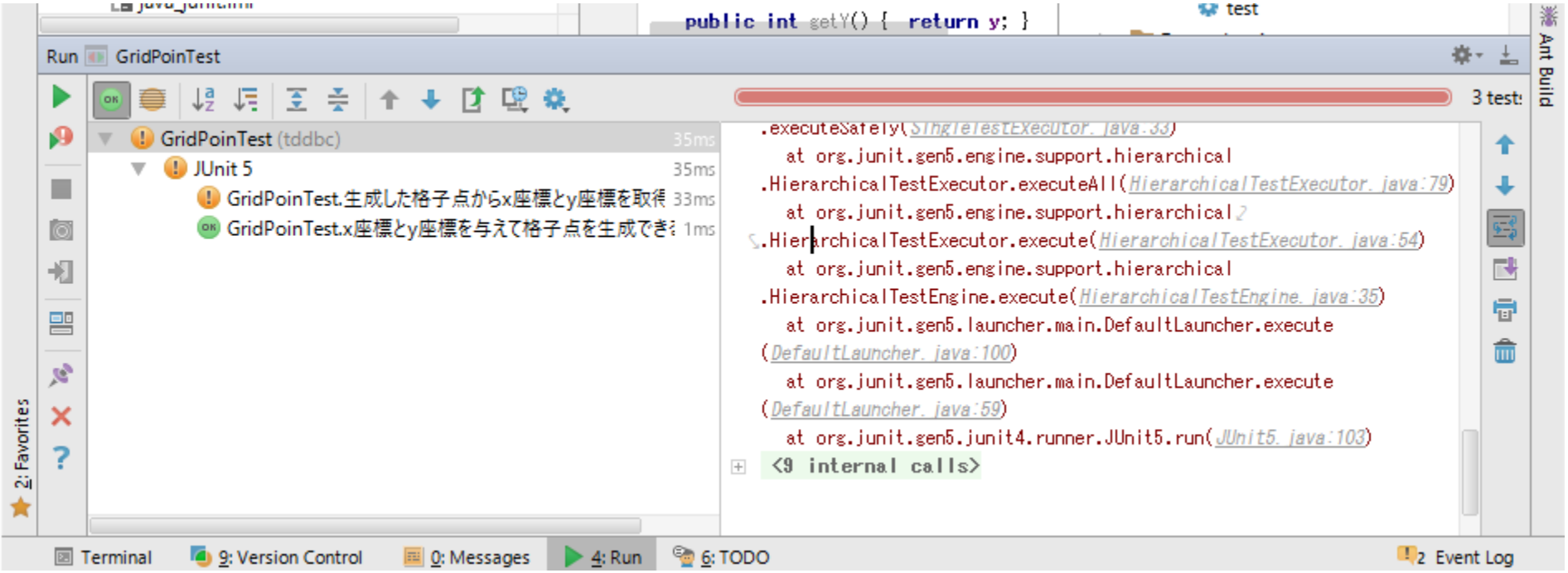
Copyright 2016 Hiroyuki Onaka

テストを追加する。

```
@RunWith(JUnit5.class)
public class GridPoinTest {

    @Test
    public void x座標とy座標を与えて格子点を生成できること(){
        // コンストラクターのテストは通常書かないですが、
        // 課題の「格子点を生成できること」と対応を取るためにあえて
        // テスト化しています
        GridPoint point = new GridPoint(4, 7);
        assertNotNull(point);
    }

    @Test
    public void 生成した格子点からx座標とy座標を取得できること(){
        GridPoint point = new GridPoint(4, 7);
        assertEquals(4, point.getX());
    }
    @Test@Disabled
    public void 生成した格子点から文字列表記を取得できること() {}
}
```

Copyright 2016 Hiroyuki Onaka

すべてのテストを実行し、新しいテストの失敗を確認する。



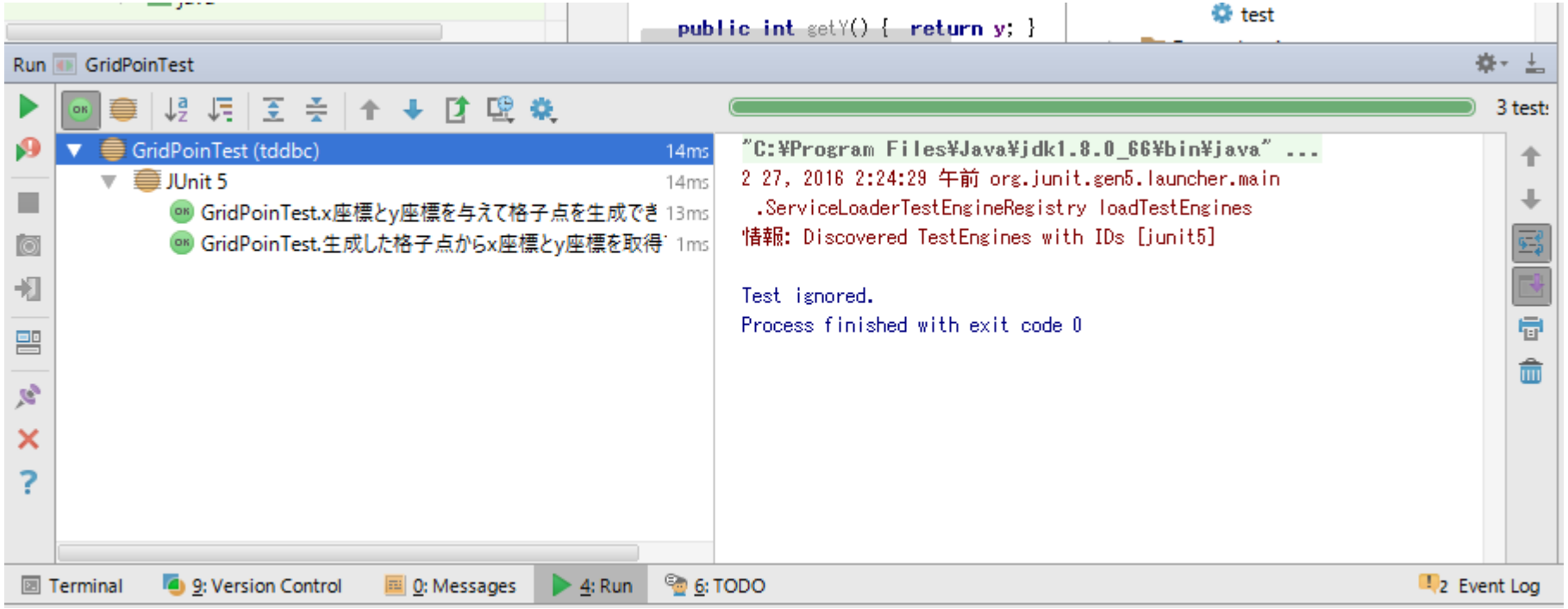
Copyright 2016 Hiroyuki Onaka

小さな修正を行う。

```
public class GridPoint {

    private int x;

    public GridPoint(int x, int y) {
        this.x = x;
    }

    public int getX() {
        return x;
    }
}
```

Copyright 2016 Hiroyuki Onaka

すべてのテストを実行し、すべての成功を確認する。



Copyright 2016 Hiroyuki Onaka

再びテストを追加する。

```
@Test
public void x座標とy座標を与えて格子点を生成できること() {
    GridPoint point = new GridPoint(4, 7);
    assertNotNull(point);


@Test
public void 生成した格子点からx座標とy座標を取得できること() {
    GridPoint point = new GridPoint(4, 7);
    assertAll(
            () -> { assertEquals(4, point.getX()); },
            () -> { assertEquals(7, point.getY()); }
    );
}
```

Copyright 2016 Hiroyuki Ohaku

再びテストを実行する。

Run GridPoinTest

3 tests

GridPoinTest (tddbc) 28ms
JUnit 5 28ms
GridPoinTest.x座標とy座標を与えて格子点を生成でき 13ms
GridPoinTest.生成した格子点からx座標とy座標を取得 15ms

```
"C:¥Program Files¥Java¥jdk1.8.0_66¥bin¥java" ...
2 27, 2016 2:28:16 午前 org.junit.gen5.launcher.main
 .ServiceLoaderTestEngineRegistry loadTestEngines
情報: Discovered TestEngines with IDs [junit5]

org.opentest4j.MultipleFailuresError: Multiple Failures (1 failure)
    expected: <7> but was: <0>
Expected :<7>
Actual   :<0>
 <Click to see difference>


    at org.junit.gen5.api.Assertions.assertAll(Assertions.java:339)
    at org.junit.gen5.api.Assertions.assertAll(Assertions.java:328)
    at tddbc.GridPoinTest.生成した格子点からx座標とy座標を取得できる
```

Terminal | 9: Version Control | 0: Messages | 4: Run | 6: TODO | 2 Event Log

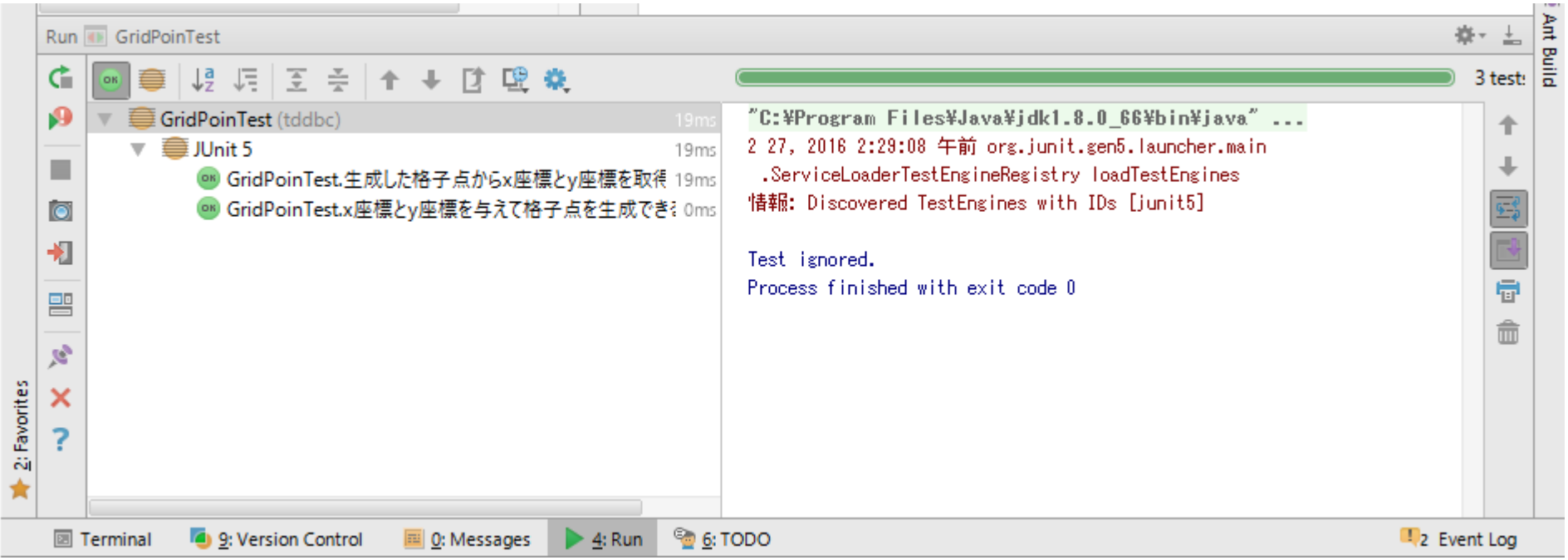
Copyright 2016 Hiroyuki Onaka

修正を行う。

```
public class GridPoint {

    private int x;

    private int y;

    public GridPoint(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public int getX() { return x; }

    public int getY() {  return y; }
}
```

TDDBC test-driven development

すべてのテストを実行し、すべての成功を確認する。



Copyright 2016 Hiroyuki Onaka

重複を取り除くためにリファクタリングを行う。

```
private GridPoint point;

@BeforeEach
public void before(){
    point = new GridPoint(4, 7);
}
@Test
public void x座標とy座標を与えて格子点を生成できること() {
    assertNotNull(point);
}
@Test
public void 生成した格子点からx座標とy座標を取得できること() {
    assertAll(
            () -> { assertEquals(4, point.getX()); },
            () -> { assertEquals(7, point.getY()); }
    );
}
```

Copyright 2016 Hiroyuki Onaka

フィードバックループ

「運転というのはね、車を正しい方向に走らせることじゃないの。常に注意を払って、こっちに行ったら少し戻して、あっちに行ったら少し戻して、そうやって軌道修正していくものよ」

これがXP のパラダイムだ。注意して、適応して、変更する。

Copyright 2016 Hiroyuki Onaka

テストファースト

Copyright 2016 Hiroyuki Onaka

TDDの観点から

TDDでは、なぜテストファーストするのか

Copyright 2016 Hiroyuki Onaka

「テストファーストしてないテストは怖い」

-@setoazusa

Copyright 2016 Hiroyuki Onaka

テストを先に書くことはどんな意味があるのか

- テストに求められる機能は、成功するとGreenになり、そうでない時にRedになること
- テストファーストしてないテストは、「まず失敗させる」ことができないため、テストとしての信頼性がさがる
- 「失敗していないテストのカバレッジは50%しかない」

Copyright 2016 Hiroyuki Onaka

思い出してみてください。

自動化されてないテストで、テスト項目書に「正しく表示されていること」と記載されていて、検証に苦労したことはありませんか？

Copyright 2016 Hiroyuki Onaka

ユニットテストが「テスト」である以上、テストの答えとなるものは先に用意されているはず

Copyright 2016 Hiroyuki Onaka

TDDではなぜテストファーストするのか

あくまでテストのあるべき姿を追求したことで、結果としてテストを最初に書くことになるイメージ。

Copyright 2016 Hiroyuki Onaka

よくある質問

「TDDって、テストを必ず先に書かなければいけないんですか？」

Copyright 2016 Hiroyuki Onaka

http://www.slideshare.net/YasutoEnjoji/20150621-asbc-pub/29

Upload

Home Technology Education More Topics My Clipboards For Uploaders Collect Le

Be the first to clip this slide

実際問題TDDしてません

- やっていないこと
  - “Red -> Green -> Refactor”
  - いわゆる３原則への準拠
- やっていること
  - “Coding -> Test -> Refactor”
  - 安心してください。テスト書いてます。

Clip slide 29 of 38

20150621 Agilesamrai Basecamp TDD事例紹介

727 views

Ad

Recommended

Agile_samurai_kpt20
Takeshi Arai
1,404 views

“総務も！！”アジャイル
ティス！
pupupopo88
4,494 views

新入社員の技術研修に
ありがちな問題を解決す
めにやったことやるべき
pupupopo88
9,305 views

テストファーストよりも大事なこと

- 「1ヶ月かけてコーディング、1ヶ月かけてテスト」のようなアンチパターンから、どれだけプログラミングとテストを連携して開発ができるようにするか
- テストによって開発が駆動されているか

Copyright 2016 Hiroyuki Onaka

- 大事なのは、システムをデリバリーするために、どうステップを踏んでいくか
- テストファーストを導入できるなら、すればよい。
- そうでないとしても、ユニットテストより上位レベルのテストの項目は先に作成するなど、テスト同士で補完していけば良い。

Copyright 2016 Hiroyuki Onaka

ただ、IT技術者として手札は多いほうがよいので、スキルとしてテストファーストできるにこしたことはないです。

Copyright 2016 Hiroyuki Onaka

なぜTDDするのか

Copyright 2016 Hiroyuki Onaka

最初に答えをいいます

「継続的インテグレーションをもたらすための核心である素早いフィードバックは、ユニットテストのカバレッジが十分にないと可能にならないのだ」

Copyright 2016 Hiroyuki Onaka

https://twitter.com/devops_borat/status/415871688870797312

「エラーが自動的に増殖するのがDevOps」

DevOps Borat
@DEVOPS_BORAT

Follow

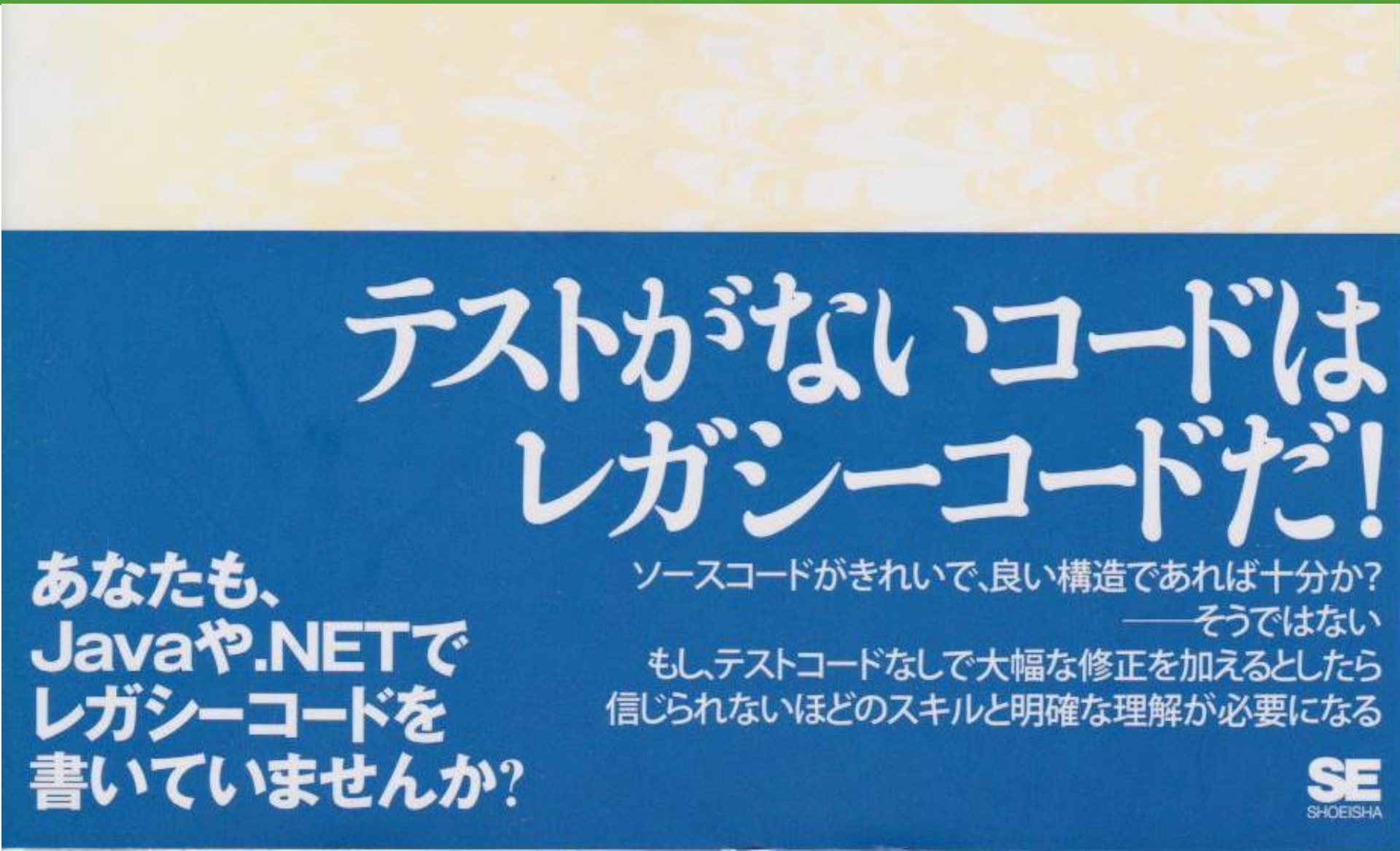
To make error is human. To propagate error to all server in automatic way is #devops.

4:55 AM - 27 Feb 2011

2,897 1,113

Copyright 2016 Hiroyuki Onaka

「テストがないコードはレガシーコードだ！」



Microsoft 「Test Early and Often」
https://msdn.microsoft.com/library/ee330950.aspx

Test Early and Often

この記事は機械翻訳されたものです。記事の文章にポインターを重ねると、翻訳されたテキストが表示されます。詳細情報 訳文 原文

Test Early and Often

Visual Studio 2012 | その他のバージョン

Catching defects as early as possible is the least expensive way to ensure software quality. Kent Beck and Cynthia Andres wrote "Here is the dilemma in software development: defects are expensive, but eliminating defects is also expensive. However, most defects end up costing more than it would have cost to prevent them." (Extreme Programming Explained: Embrace Change) Best practices and tools can help your team to minimize the cost of preventing and fixing defects by maintaining the quality of your project throughout its lifecycle.

Your team can more accurately gauge the quality of your project at any time if you find defects, fix defects, and verify fixes as you go along. By testing often, your team and stakeholders can remain aware of the current state of the code and make informed decisions throughout the project. Ultimately, you should be able to answer the question "Can we release?" and understand the implications for the people who use the software.

This topic recommends the following practices:

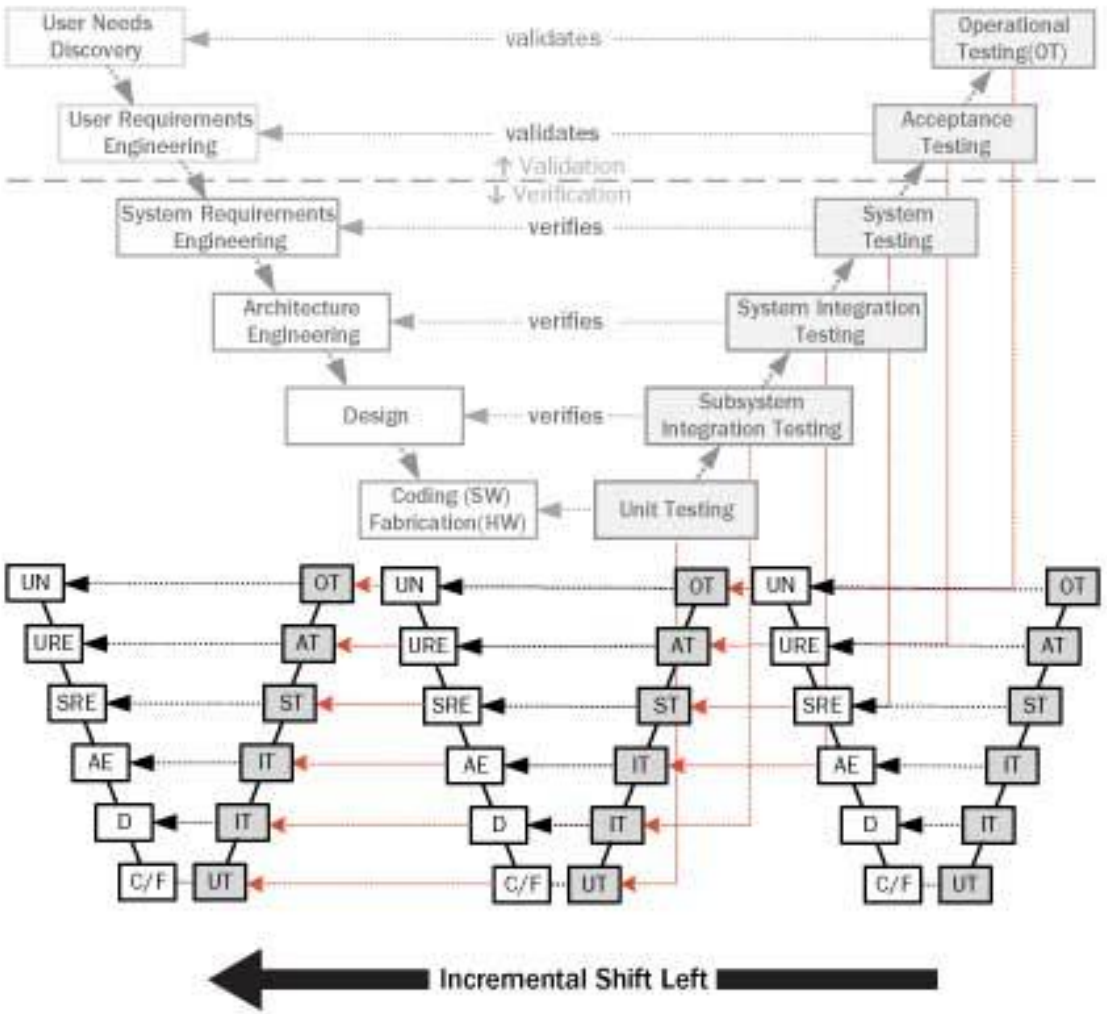
- Create a set of automated unit tests for each class and for the API of every major component. Writing unit tests should take about 40% of team members' time. For more information, see Creating Automated Tests Using

この記事の内容

In this topic
Test Strategy
Test Planning
Acceptance Testing
Unit Testing
Test-Driven Development and Testing Early
Manual versus Automated Testing
Reporting the Test Results

By DonFiresmith (Own work) [CC BY-SA 4.0 (http://creativecommons.org/licenses/by-sa/4.0)], via Wikimedia Commons
https://en.wikipedia.org/wiki/Shift_left_testing

Shift left Testing



とはいうものの…

- TDD=テスト自動化ではない

Copyright 2016 Hiroyuki Onaka

TDDが目指すもの

「動作するきれいなコード」、ロン・ジェフリーズのこの簡潔な言葉は、TDD（テスト駆動開発）の目標である。動作するきれいなコードは、あらゆる理由で価値がある。

Copyright 2016 Hiroyuki Onaka

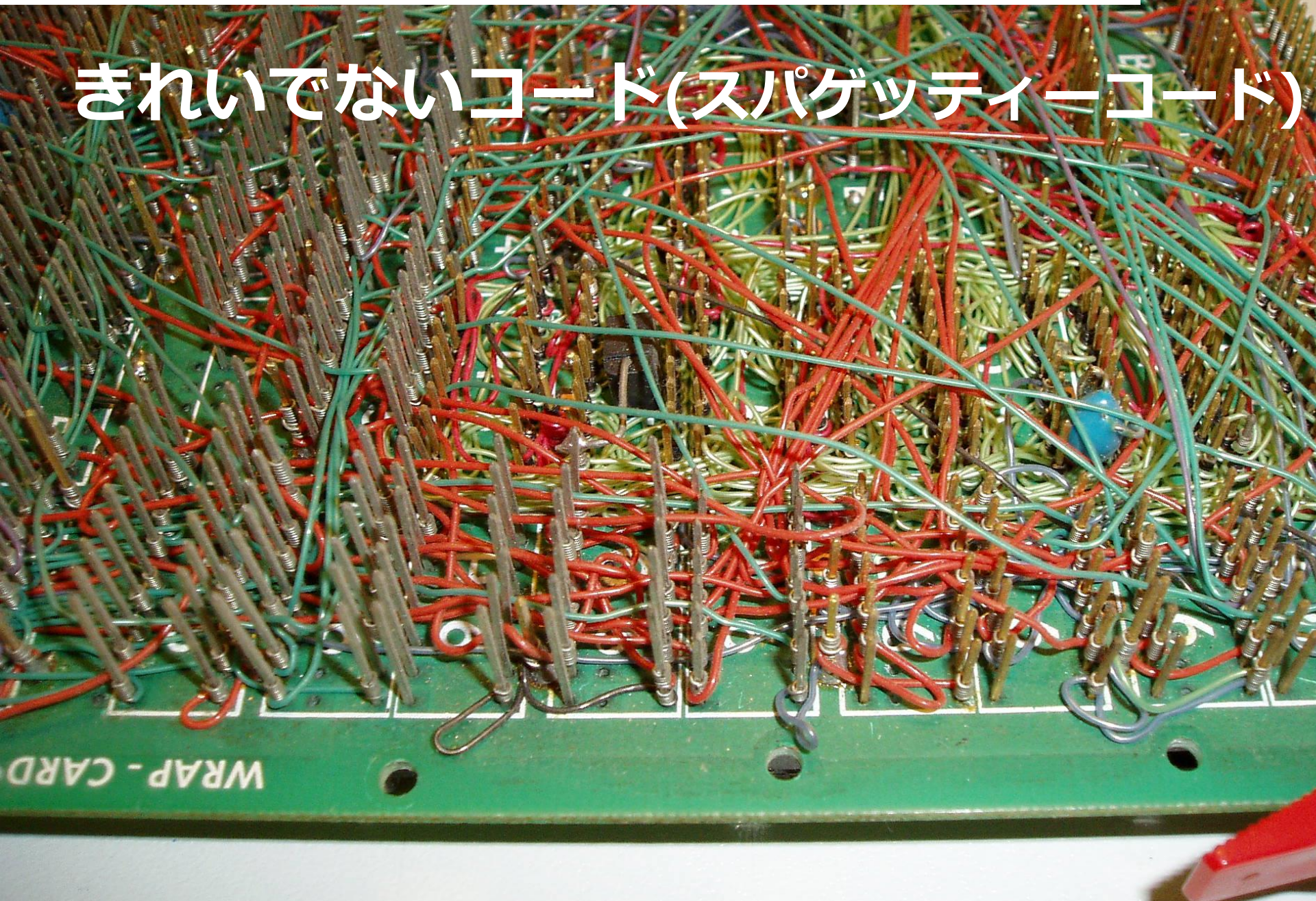
「動作するきれいなコード」

「きれいなコード」とは？

Copyright 2016 Hiroyuki Onaka

By Wikinaut (Own work (own photo)) [GFDL (http://www.gnu.org/copyleft/fdl.html) or CC BY-SA 3.0 (http://creativecommons.org/licenses/by-sa/3.0)], via Wikimedia Commons
https://ja.wikipedia.org/wiki/スパゲッティプログラム

きれいでないコード(スパゲッティーコード)



では「きれいなコード」とは

…なんだ？

Copyright 2016 Hiroyuki Onaka

「動作するきれいなコード」を英語で言うと、

「Clean code that works」

Copyright 2016 Hiroyuki Onaka

＿人人人人人人人人人人人人人＿
＞　Beautifulじゃなかった　＜
￣Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^

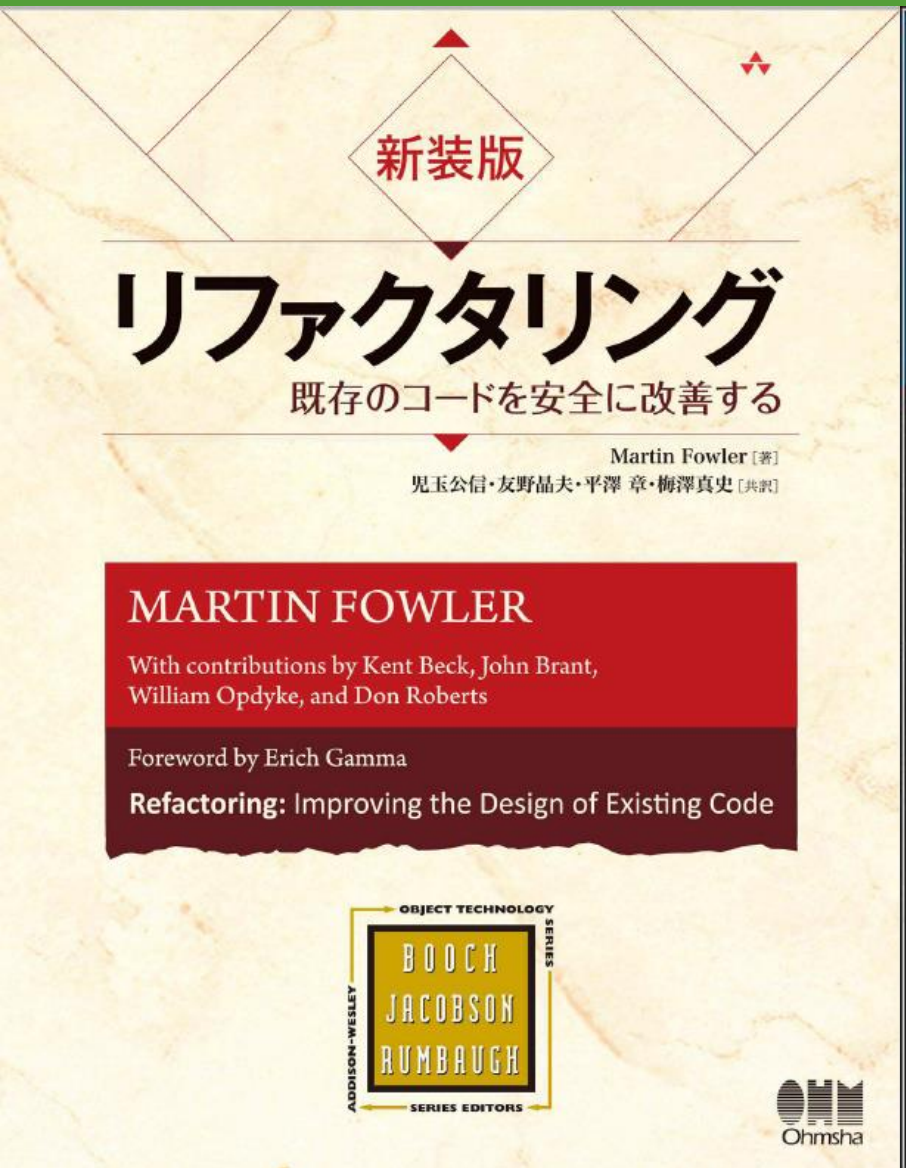
Copyright 2016 Hiroyuki Onaka

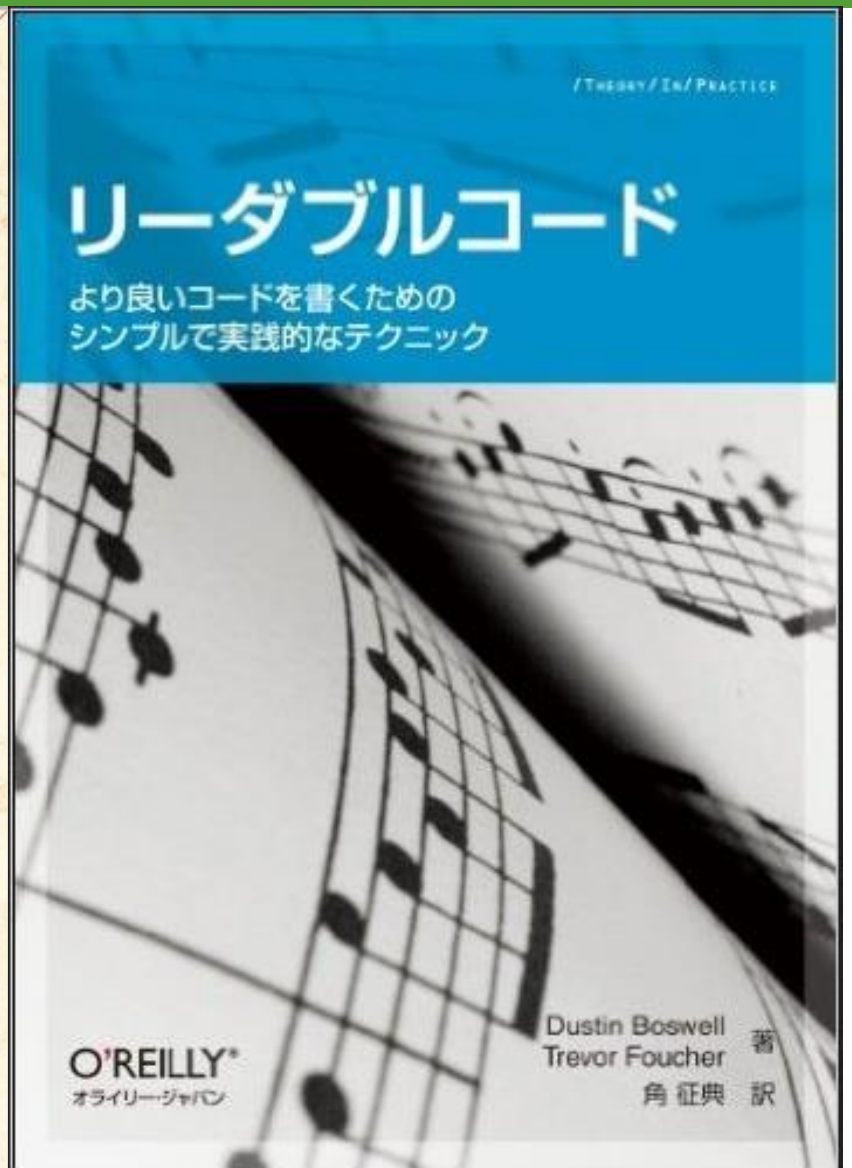
とはいっても、「クリーン」にも色々ありまして

- 複雑度
- メソッド／関数の行数
- コードスタイル
- 命名規則
- etc…

Copyright 2016 Hiroyuki Onaka

「クリーンなコード」についてはこちらを





TDDBC test-driven development
CC BY

どうテストするか

Copyright 2016 Hiroyuki Onaka

http://gihyo.jp/dev/serial/01/tdd/0010

「不安をテストで表現する」

か，どういう単位でテストを書けばいいのかとかで詰まってしまう」という質問をいただきましたね。

私は実は，明確にこういう単位でテストを書くべしという決まりはないと思っています。

不安をテストで表現する

では，私がどういうときに小さいテストを書いているかというと，私は「不安」を最小単位の基準としています。不安を手がかりにしてテストを書きます。

私たちプログラマの手を止めるものは何でしょうか。私は「不安」だと思っています。「もしかしたら」という感情ですね。「もしかしたら，自分の書いているコードは間違っているかもしれない」「もしかしたら，ライブラリの使い方が正しくないかもしれない…」。

私は不安に対してテストを書いていますから，自分がこれから書くコー

SAP HPC 徹底活用!!
の比較，また，実際に無料
て試用するまでの手順を紹

イベント

- SAP HANA Cloud Forum Tokyo（9/
- Seasar Conference 2015（9/26）
- PHPカンファレンス2015（10/3）
- java-ja.OSS（10/5）
- PyCon JP 2015（10/9〜12）
- 第5回「アプリ甲子園」決勝大会（10/
- CODE BLUE 2015（10/28, 29）
- TeXユーザの集い2015（11/7）
- 東京Node学園祭2015（11/7）
- OpenID Summit Tokyo 2015（11/1
- RubyWorld Conference 2015（11/
- AVTOKYO2015（11/14）

@t_wada曰く

「［動画で解説］和田卓人の“テスト駆動開発”講座　第10回　テストの最小単位は不安」
http://gihyo.jp/dev/serial/01/tdd/0010

私たちプログラマの手を止めるものは何でしょうか。私は「不安」だと思っています。「もしかしたら」という感情ですね。「もしかしたら，自分の書いているコードは間違っているかもしれない」「もしかしたら，ライブラリの使い方が正しくないかもしれない…」。（略）

だから，これから書くコードに対して，if文があるだろうなとか，ループがあるとか，正規表現使わなきゃいけないなとか，そういった自分自身に対する不安，これから書くことに対する不安に対して，テストを書いていきます。

Copyright 2016 Hiroyuki Onaka

- プログラマーとしての「不安」
- データベース技術者としての「不安」
- インフラ担当としての「不安」

Copyright 2016 Hiroyuki Onaka

- お互いが不安に感じていることを認める。
- その人の弱さを認める。それを品質保証の出発点にする。

Copyright 2016 Hiroyuki Onaka

James O. Coplien , Neil B. Harriosn (著), 和智右桂 (翻訳) 「組織パターン」

「信頼で結ばれた共同体」

もう一つ、よく聞く話

「TDDでテストすれば、他のテストはいらないんですか？」

Copyright 2016 Hiroyuki Onaka

開発者と品質保証の関係

- 開発者テストのレベルで「開発者の思ったとおりに動く」ことが保証されているから、品質保証のテストは品質の検証に注力できる
- 品質保証のテストが後詰めとして控えているから、開発者テストはプログラムの中の勘所を押さえることに注力できる

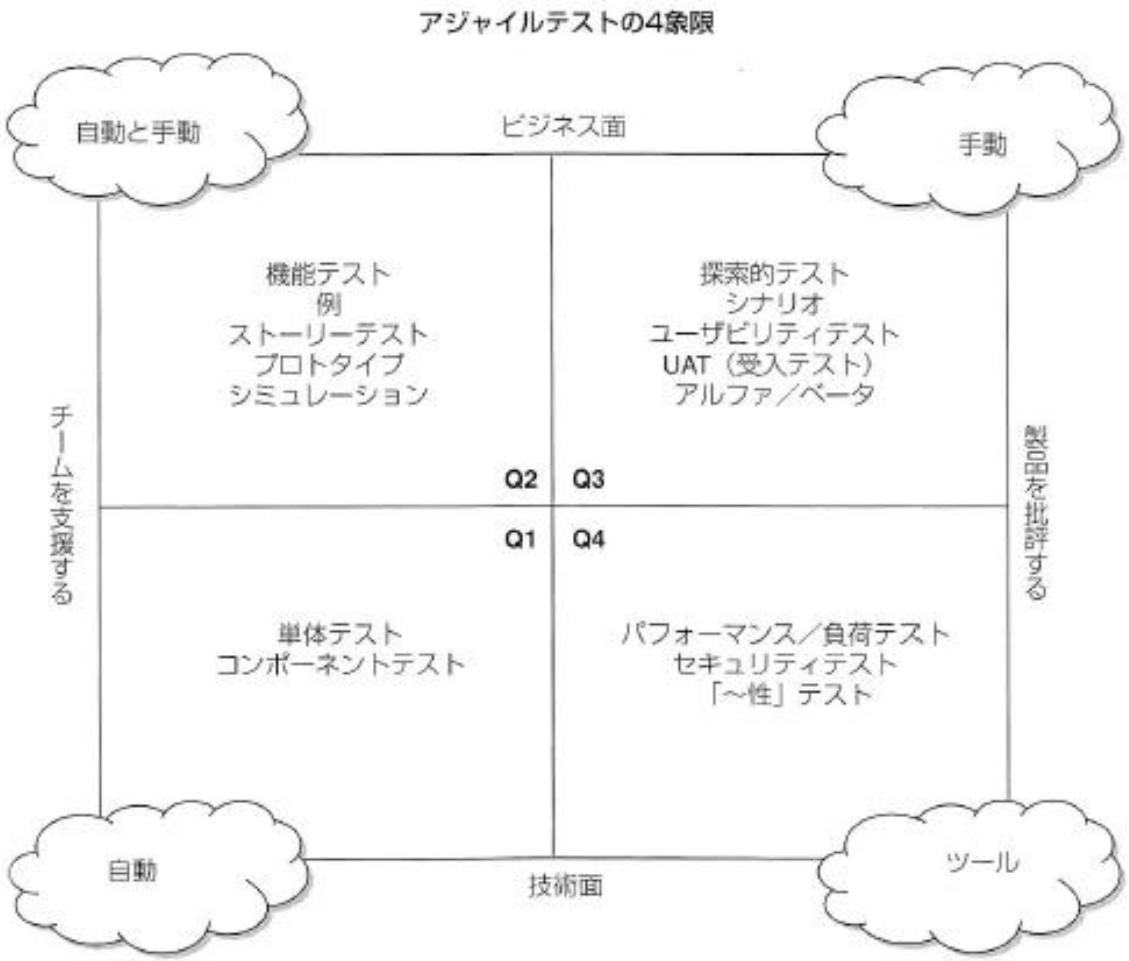
Copyright 2016 Hiroyuki Onaka

製品品質モデル(JIS X 25010)

- 機能適合性
- 性能効率性
- 互換性
- 使用性
- 信頼性
- セキュリティ
- 保守性
- 移植性

Copyright 2016 Hiroyuki Onaka

アジャイルテストの四象限



Copyright 2016 Hiroyuki Onaka

Steve Freeman, Nat Pryce「Growing Object-Oriented Software, Guided by Tests」
(邦訳「実践テスト駆動開発」)

「二重のフィードバックループ」

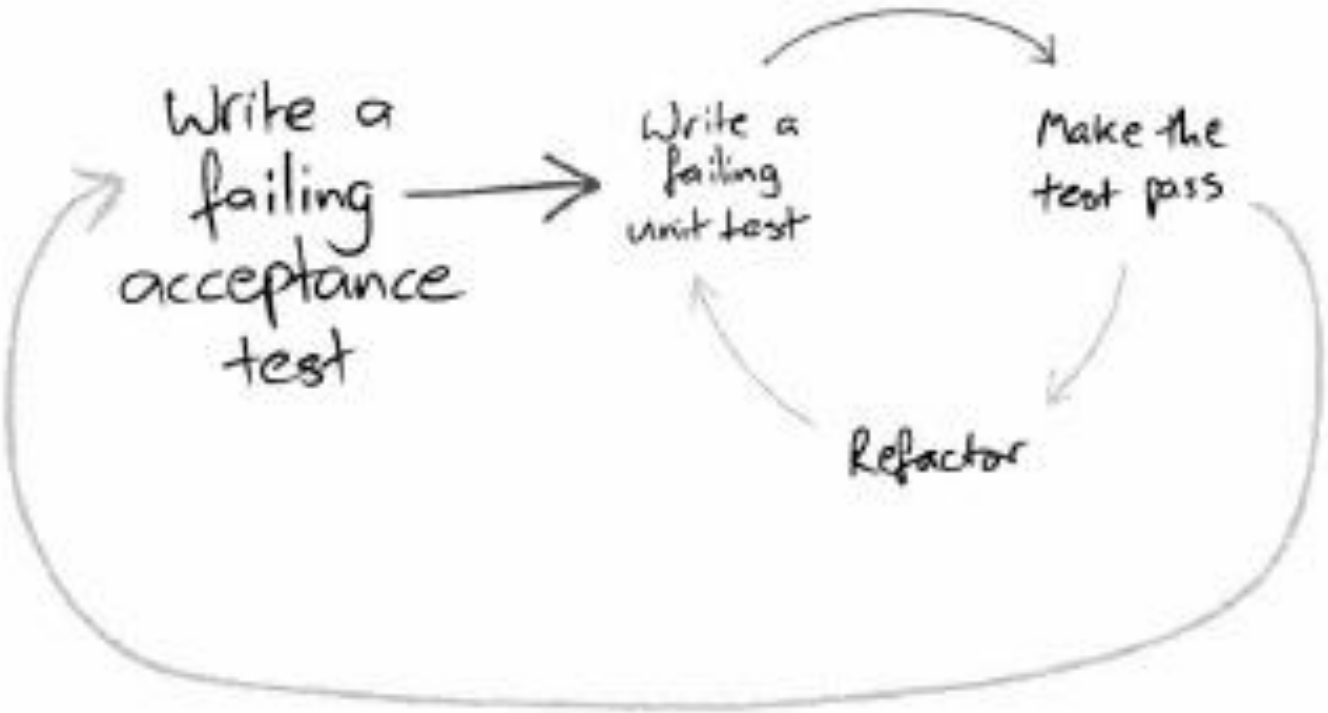


Figure 1.2 *Inner and outer feedback loops in TDD*

Copyright 2016 Hiroyuki Onaka

BDD(Behavior-Driven Development)

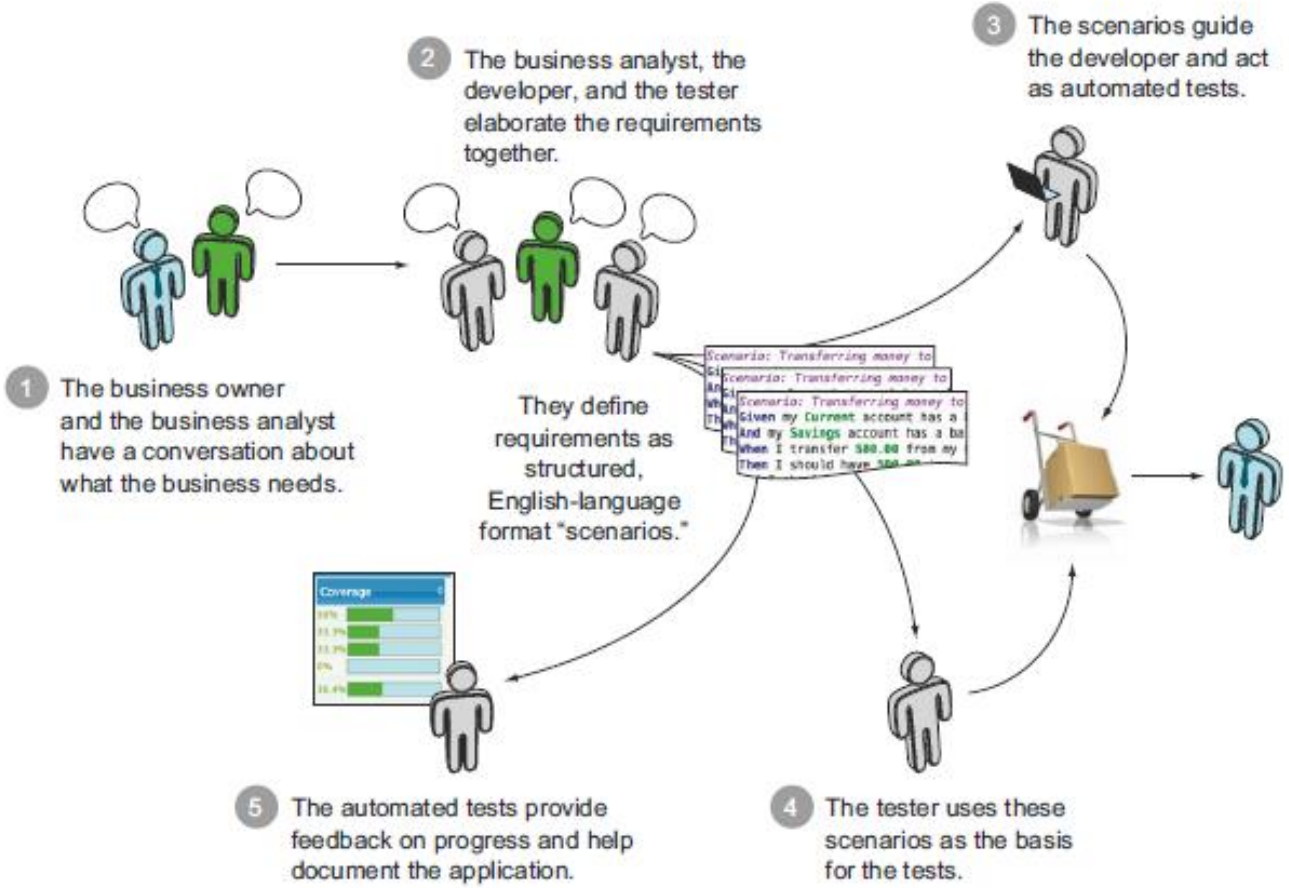


Figure 1.2 BDD uses conversations around examples, expressed in a form that can be easily automated, to reduce lost information and misunderstandings.

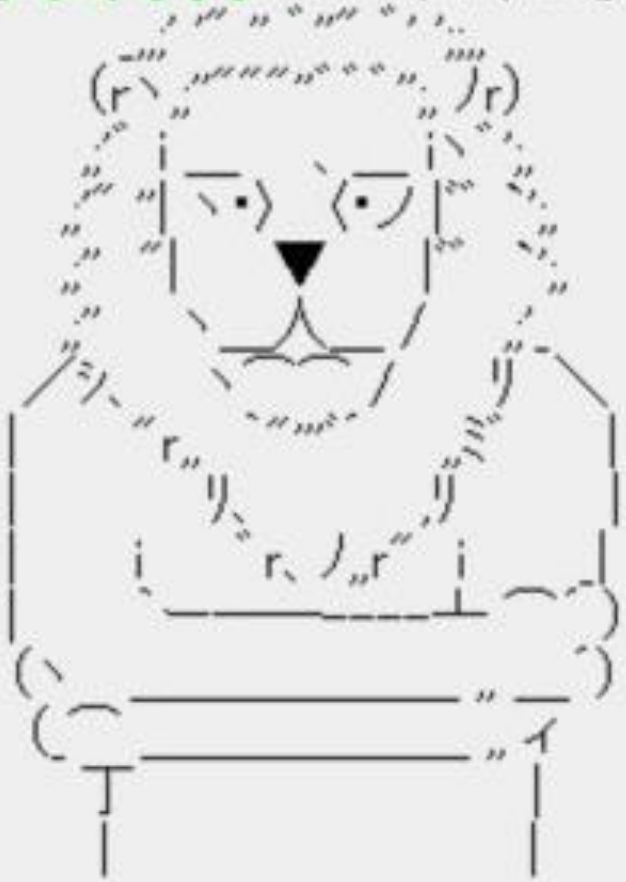
Copyright 2016 Hiroyuki Onaka

なぜテストするか

Copyright 2016 Hiroyuki Onaka

TDDはプログラマーの義務？

名無し募集中。。。 :2011/12/06(火) 18:32:24.37 0



テスト書いてないとかお前それ
@t_wadaの前でも
同じこと言えんの?

だがちょっと待って欲しい

「テストを書く／書かない」ということは、道義的な責任の問題ではない

Copyright 2016 Hiroyuki Onaka

「テスト」と責任の関係

- テストという工程は、その性格上、品質保証や、成果物への責任と強い関係をもっている
- 「テストを書くべきだ」という論に明快に異を唱えられる人は少ない

Copyright 2016 Hiroyuki Onaka

- 開発プロセスの望ましい姿を論じていたはずが、道義的な責任の問題になってしまう
- テストを書く人と書かない人の間の感情的な対立は、何も生み出さない

Copyright 2016 Hiroyuki Onaka

http://b.hatena.ne.jp/entry/232721484/comment/t-wada

「TDDやれば偉いわけじゃない」

はてなブックマーク > テクノロジー > TDDを諦めることと、RSpecをやめること - 高柴ラボ > t-wadaのコ

RSpec の話と TDD の話が混ざっている印象。個別のツールや狭義の TDD (TDD 三原則とか) に拘ったり怯えたりするのは生産的でないと思う。TDD やれば偉いわけじゃないし、やらないと悪いわけでもない。

TDD， rspec

t-wada のコメント 2014/10/20 10:51

★ スターを付ける ブログを書く ページに埋め込む

幻想を捨てる

- カバレッジ100%
- ユニットテストを書けばバグがなくなる

Copyright 2016 Hiroyuki Onaka

「テストを書く」ということは、継続的インテグレーション(CI)や静的解析、コードレビューやペアプログラミングなど、チーム開発のためのプラクティスと連携して、初めてその力を発揮する。

Copyright 2016 Hiroyuki Onaka

TDDが銀の弾丸でないなら、なぜTDDするのか

その前に、ちょっと世の中の様子を見てみよう

Copyright 2016 Hiroyuki Onaka

https://twitter.com/yotchang4s/status/699030352686256129

よっちゃん@プロ社畜
@yotchang4s

Follow

やはり自分以外全会一致でJUnitでは無くEclipseのデバッガを一行一行ステップ実行した画面キャプチャをExcelに貼り付ける事になりそう。

9:39 AM - 15 Feb 2016

148 71

Copyright 2016 Hiroyuki Onaka

https://www.google.co.jp/search?q=%E3%82%B7%E3%82%B9%E3%83%86%E3%83%A0%E9%9A%9C%E5%AE%B3&tbm=news

Google　システム障害　ログイン

すべて　ニュース　画像　ショッピング　動画　もっと見る　検索ツール

約 61,700 件（0.18 秒）

オンラインシステム障害 大分銀も

毎日新聞 - 10 時間前

西日本シティ銀行（福岡市）は15日、同行が使用するオンラインシステムに障害が起きて、一部のATM（現金自動受払機）が使えない状況になったと明らかにした。同じシステムを導入している大分銀行（大分市）も一部で使えなくなった。

地銀7行でATM障害 西日本シティ銀など

日本経済新聞 - 3 時間前

詳細を見る（他 5 件の記事）

カナロコ（神奈川新…

マイナンバーでシステム障害 影響長引く可能性も

朝日新聞 - 2016/01/19

今月から役所の手続きで使われ始めたマイナンバー（社会保障・税番号）で18～19日、システム障害が全国的に起きた。市区町村で希望者に無料で配る「個人番号カード」が一時的に発行できず、住民の情報もやりとりしづらくなった。カードの …

マイナンバーの個人番号カード交付できず システム障害

日本経済新聞 - 2016/01/19

詳細を見る（他 68 件の記事）

システム障害の疑義が寄せられた時の対応は？ - IPA教訓集

Security NEXT - 2016/02/08

IPAでは同機構が情報処理システムの障害事例や対策手法を体系化し、信頼できるシステムの構築に役立つ情報を取りまとめ、「情報処理システム高信頼化教訓集」として公開しており、今回「ガバナンス、マネジメント領域」に「システム動作の …

https://twitter.com/setoazusa/status/650250873017204736

setoazusa(せとあず♂)
@setoazusa

Follow

「システム障害」でググったらそのタイミングでWindows10がBSODで再起動した…

7:07 PM - 3 Oct 2015

5 4

Copyright 2016 Hiroyuki Onaka

#笑ってはいけないSIer

トップ | ライブ | アカウント | 画像 | 動画 | その他のオプション

キッがリツイートしました

てぃーびー @tbpgr · 2月24日
Lv6. 自称40歳だけどどうみても60歳を越えていて、茶髪のヅラからカツラの紐がはみ出ていて、ずーっと独り言と舌打ちをしていて日々SVNリポジトリを破壊し、おねぇ系のチームリーダーの怒りをかって退場させられたおじいちゃんプログラマ
#笑ってはいけないSIer

2 3

loowel/しがない職業ぷろぐらま @ra_moo_t · 2月24日
git使いたい、というと多分「マージ機能とやらが信頼できるとは言えない」と言われそうだなぁ。 #笑ってはいけないSIer

6 7

ドローン守銭奴（みうらかずひと）がリツイートしました

loowel/しがない職業ぷろぐらま @ra_moo_t · 2月24日
git使ってくれている人から居なくなっていく現象を何と名付けたら良いですか #笑ってはいけないSIer

我々はなぜここにいるのか

- 生活のため？
- お金のため？
- プロだから？
- 技術者だから？

Copyright 2016 Hiroyuki Onaka

なぜTDDするのか

私の場合…

Copyright 2016 Hiroyuki Onaka

信頼できる成果物のために

「技術力は信頼関係につながる。作業を正確に見積もり、最初から品質の高いものを届け、高速なフィードバックループを構築すれば、あなたは信頼されるパートナーになれる。」

Copyright 2016 Hiroyuki Onaka

強いチームとは

メンバーを集めた「プロジェクト」を、メンバーのバックグラウンドを尊重できる「コミュニティ」に出来るか

Copyright 2016 Hiroyuki Onaka

「君が質の高いソフトウェアを届けることは誰にも止められない。君が現場に立って、お客さんに向けてプロジェクトの状況と、プロジェクトに必要なことを誠実に伝えることも誰にも止められないんだ。」

Copyright 2016 Hiroyuki Onaka

ありがとうございました！

- 大中浩行(Onaka,Hiroyuki)
- @setoazusa
- グロースエクスパートナーズ株式会社
  アーキテクチャソリューション部
  テクニカルリード
- http://blog.fieldnotes.jp/

Copyright 2016 Hiroyuki Onaka